

Visão Computacional com CNNs e Transformers — Aula 2

Tudo o Que Você Precisa É de Atenção: Tokenização BPE, Self-Attention e o Bloco Transformer Completo

Etapa / Aula 02

Prof. Allan Spadini • Pós-Graduação em Inteligência Artificial & Visão Computacional

✦ Recap: Segmentação U-Net

✦ Completação & BPE Tokenizer

✦ O Que São Embeddings & Semântica

✦ A Matriz Look-up Table & Contexto

✦ Dot-Product Attention (Q, K, V)

✦ O Dilema da Ordem & Embeddings de Posição

✦ Máscara Causal & Multi-Head

✦ Arquitetura Seq2Seq para Tradução

✦ Bloco Transformer, Pre-LN & ViT

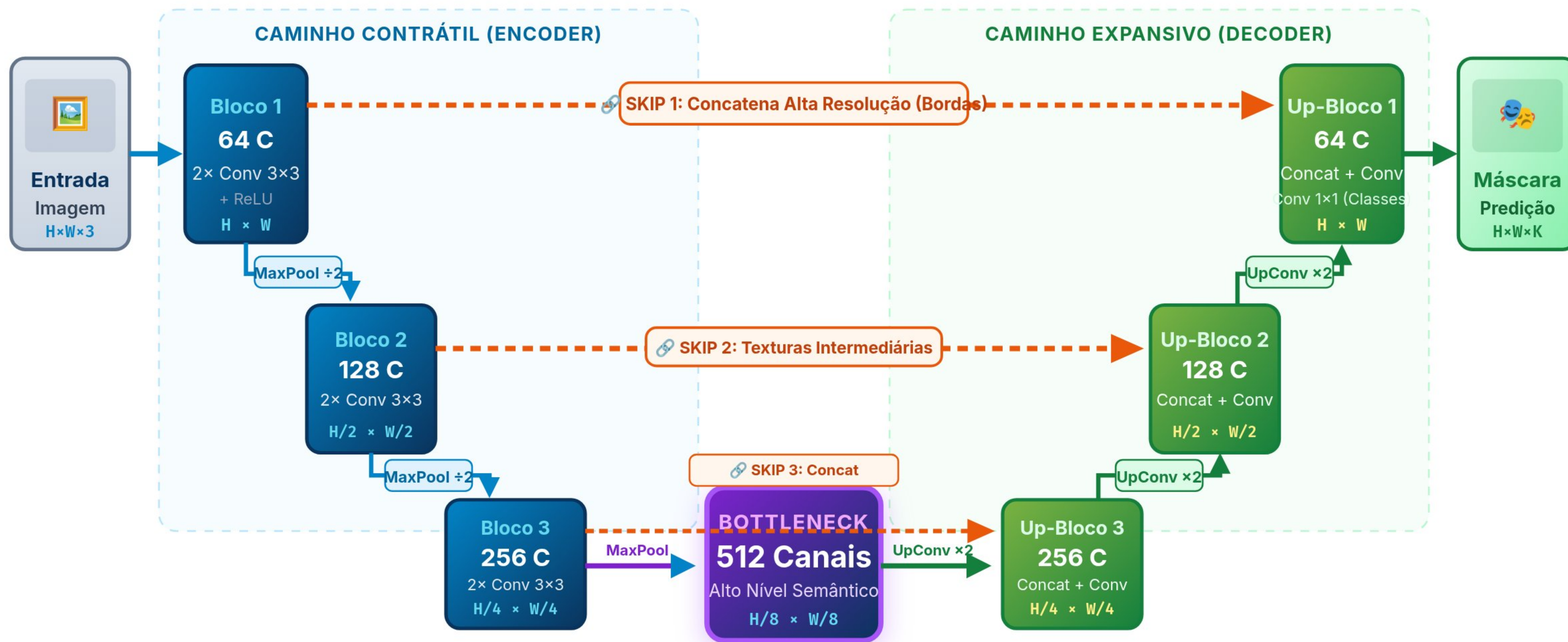
Recapitulação: A Anatomia da Segmentação Semântica

U-Net, Conexões Skip Densas e a Transição da Grade de Pixels para Sequências

Arquitetura da U-Net: Encoder-Decoder com Conexões de Atalho Densas (Skip Connections)

Arquitetura U-Net

CNN Local vs Transformer Global





O Paradigma da Completação: Previsão do Próximo Token

Modelagem de Linguagem Autorregressiva: $P(x_t \mid x_1, \dots, x_{t-1})$

✦ O Paradigma Fundamental: Predição Autorregressiva do Próximo Token

▶ Avançar Passo Autorregressivo (1/3)

1. SEQUÊNCIA DE ENTRADA (PROMPT CONTEXTUAL):

Tudo o que você precisa é de [?]

BLOCOS DE ATENÇÃO CAUSAL (TRANSFORMER)

Processa tokens com Self-Attention e projeta $d_{\text{model}} \rightarrow |V_{\text{vocab}}|$

Softmax(Logits)

2. AMOSTRAGEM / SELEÇÃO DO PRÓXIMO TOKEN:

✓ Token Escolhido (ArgMax / Top-p):
"amor" (Probabilidade: 94.2%)

💡 O modelo recebe o prompt inicial da canção dos Beatles e calcula a distribuição de probabilidade sobre todo o vocabulário para a posição seguinte.

DISTRIBUIÇÃO SOFTMAX NO VOCABULÁRIO $P(X_T \mid X_{<T})$



Por que isso importa?

Toda tarefa de NLP moderna (geração de texto, tradução, código, e até visão como geração de sequências) é formulada como a previsão sequencial do próximo token condicionada ao histórico anterior.

O Pipeline Completo de Processamento em Transformers

Da String Bruta à Distribuição de Probabilidades no Vocabulário

1. Tokenização BPE

- Decompõe o texto bruto em subpalavras estatísticas e bytes.
- Mapeia cada pedaço para um índice inteiro único no vocabulário.
- Elimina completamente o problema de palavras fora do vocabulário (OOV).

2. Embeddings & Posição

- Converte os IDs inteiros em vetores densos contínuos em $\mathbb{R}^{d_{\text{model}}}$.
- Injeta sinal de ordem espacial/temporal via Embeddings de Posição.
- Forma a matriz de entrada do modelo com formato [Batch, T, d_{model}].

3. Atenção Multi-Head

- Múltiplas cabeças projetam o sinal em Queries, Keys e Values.
- Calcula matrizes de afinidade direta entre todos os pares de tokens.
- Aplica máscara causal (se autorregressivo) e conexões residuais.

4. MLP & Head de Saída

- Rede Feed-Forward com expansão 4x atua como memória associativa.
- Camada Linear projeta os vetores de volta para a dimensão do vocabulário $|V|$.
- Softmax gera as probabilidades finais de completção ou classe.

 Tudo o que você precisa é de atenção: cada componente do pipeline desempenha um papel matemático estrito na transformação de símbolos em inteligência.

Tokenização Subword: O Algoritmo Byte-Pair Encoding (BPE)

Superando o Dilema entre Caracteres Isolados e Palavras Inteiras

Tokenização Subword: O Algoritmo Byte-Pair Encoding (BPE)

[Mecânica do Algoritmo](#)[Níveis de Tokenização](#)

EVOLUÇÃO DAS FUSÕES NO BPE (EXEMPLO DA CANÇÃO DOS BEATLES)

1. Inicialização

[Passo 1](#)

T u d o _ o _ q u e _ v o c ê _ p r e c i s a _ é _ d e _ a m o r

Cada caractere ou byte inicial é um token individual no vocabulário base (256 bytes).

2. Par Mais Frequente

[Passo 2](#)

T u d o _ o _ q u e _ v o c ê _ p r e c i s a _ é _ d e _ [a m] o r

O algoritmo conta a frequência de todos os pares adjacentes no corpus. O par ("a", "m") se funde em "am".

3. Fusão Sucessiva

[Passo 3](#)

T u d o _ o _ q u e _ v o c ê _ p r e c i s a _ é _ d e _ [a m o r]

O par ("am", "or") se funde no token único "amor" (ID 8954). O mesmo ocorre com "você" e "precisa".

4. Vocabulário Final

[Passo 4](#)

[Tudo] [o] [que] [você] [precisa] [é] [de] [amor]

Subwords frequentes viram tokens únicos. Palavras raras ou neologismos são decompostos em sub-pedaços sem erro de OOV!



Por que o BPE Venceu?

- **Zero Out-of-Vocabulary (OOV):** Qualquer caractere desconhecido ou emoji pode ser decomposto nos seus bytes UTF-8 individuais.
- **Vocabulário Fixo:** Tamanho controlado (ex: GPT-2 usa 50.257 tokens; GPT-4/tiktoken usa 100k tokens).
- **Compressão Equilibrada:** Palavras comuns usam 1 único token; palavras complexas ou derivadas usam 2 a 3 subwords.
- **Espaço em Branco é Token:** Espaços (ex: Glove ou _amor) são codificados explicitamente junto com a palavra.



Tiktoken / GPT-2: A biblioteca padrão para tokenização em PyTorch rápida em Rust/C++ que veremos no laboratório a seguir!



Laboratório Interativo 1: Explorador de Tokenização BPE

Experimente com Letras dos Beatles e Inspeção IDs, Bytes e Embeddings

 Laboratório BPE: Tokenização e Mapeamento de IDs

Exemplos da Canção: **Refrão Principal** Refrão em Inglês Verso Complexo 1 Verso Complexo 2 Final Icônico

Tudo o que você precisa é de amor

SEQUÊNCIA DE TOKENS SUBWORD GERADA (16 TOKENS):

Clique em um token para inspecionar

Tudo #5728  #1032 o #1111  #1032 que #12807  #1032 você #7808  #1032

prec #32900 isa #5053  #1032 é #1233  #1032 de #4201  #1032 amor #33868

Caracteres	Palavras	Tokens BPE	Tokens / Palavra
33	8	16	2.00x

INSPECTOR DE TOKEN & PROJEÇÃO PARA EMBEDDING



Clique em qualquer token à esquerda
Para visualizar os bytes UTF-8, o ID no vocabulário e a projeção vetorial no embedding.

O Que São Embeddings? Do One-Hot ao Hiperespaço Semântico

Hipótese Distribucional de Firth, Similaridade Cosseno e o Treinamento End-to-End

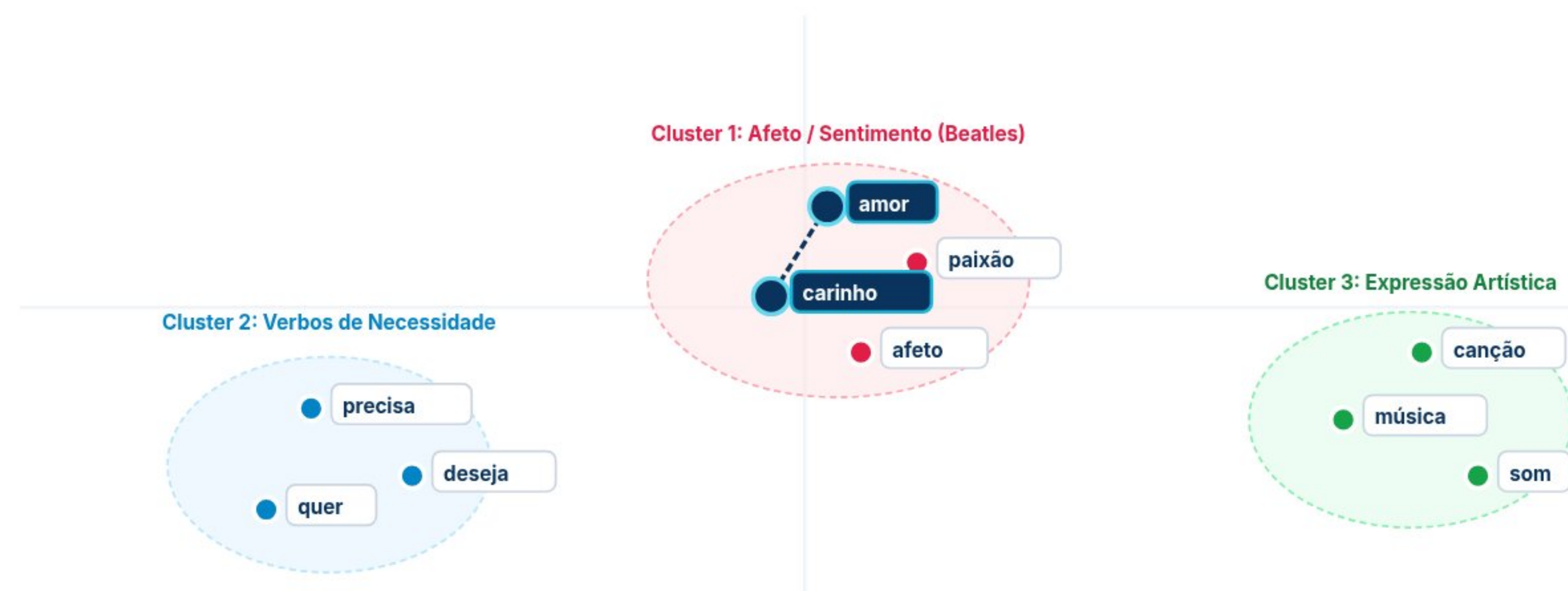
O Que São Embeddings? O Hiperespaço Semântico Contínuo

Hipótese Distribucional (Firth, 1957)

Treinado End-to-End via Backpropagation

PROJEÇÃO 2D DO ESPAÇO LATENTE ($\mathbb{R}^{768} \rightarrow \mathbb{R}^2$ VIA T-SNE / PCA)

(Clique em duas palavras para calcular o Cosseno)



Comparador Semântico: "amor" vs "carinho" Similaridade Cosseno: $\cos(\theta) = 0.998$ 🔥 (Próximos)

1. É um modelo separado? Como aprende?

Nos Transformers modernos, **NÃO** é um modelo externo! É simplesmente a primeira camada de pesos (`nn.Embedding`) treinada **end-to-end** com a rede.

Hipótese Distribucional (Firth, 1957): Palavras que aparecem nos mesmos contextos recebem atualizações de gradiente parecidas pelo Backpropagation, convergindo para o mesmo cone no espaço vetorial!

2. Por Que Não Usar One-Hot Encoding?

❌ One-Hot (50.257-d)

- Vetores esparsos gigantes.
- **Ortogonais**: $u \cdot v = 0$.
- "amor" e "carinho" parecem tão distantes quanto "amor" e "trator".

✅ Dense Embed (768-d)

- Vetores densos compactos.
- **Semântica Contínua**: $\cos(\theta)$ reflete afinidade.
- **Álgebra Vetorial**: rei - homem + mulher $\approx$ rainha.

🌟 **Generalização**: O aprendizado sobre "amor" se transfere automaticamente para "afeto" e "carinho"!



A Matriz Look-up Table e o Tensor 3D de Saída

Mapeamento $WE \in \mathbb{R}^{|V| \times d_{\text{model}}}$, Leitura $O(1)$ e o Tensor $[B, T, d_{\text{model}}]$

A Matriz de Embeddings na Arquitetura: Da Look-up Table ao Tensor 3D

PyTorch: `nn.Embedding(50257, 768)`

Shape: $[Batch, Sequence, d_{\text{model}}]$

Acesso $O(1)$ em VRAM

1. Sequência Discreta (T=8)

t=0	"Tudo"	ID:1420
t=1	"o"	ID:284
t=2	"que"	ID:853
t=3	"você"	ID:4920
t=4	"precisa"	ID:1120
t=5	"é"	ID:350
t=6	"de"	ID:290
t=7	"amor"	ID:8954

Tensor Inteiro $[B=16, T=8]$

$O(1)$

2. Matriz Look-up Table ($W_E \in \mathbb{R}^{50.257 \times 768}$)

Índice V	dim 0	dim 1	dim 2	...	dim 767
Linha 0	+0.021	-0.114	+0.342	...	-0.052
Linha 284	-0.082	+0.219	-0.145	...	+0.603
⋮ (50.000+ vetores aprendidos na memória GPU)					
Linha 8954 ("amor")	+0.882	+0.310	-0.149	...	+0.592
⋮					
Linha 50256	-0.043	+0.721	-0.198	...	+0.119

Operação PyTorch: `X_emb = nn.Embedding(50257, 768)(input_ids)`
Lookup direto por ponteiro de memória — Zero multiplicação matricial $O(|V|)$
50.257 linhas $\times$ 768 floats $\times$ 4 bytes $\approx$ 154 MB de parâmetros treináveis

Empilha

3. Tensor Contínuo 3D (X_{emb})

T = 8 tokens (Janela)

Batch Atual #0		$[B, T, d]$
t=0 ("Tudo")	[+0.124, -0.451, ...]	
t=1 ("o")	[-0.082, +0.219, ...]	
t=2 ("que")	[+0.341, +0.012, ...]	
t=3 ("você")	[-0.215, +0.672, ...]	
t=4 ("precisa")	[+0.512, -0.334, ...]	
t=5 ("é")	[-0.104, +0.088, ...]	
t=6 ("de")	[+0.043, -0.198, ...]	
t=7 ("amor")	→ [768 floats]	

← $d_{\text{model}} = 768$ dimensões contínuas →

⚠ **Próximo Passo Crítico:**
Até aqui, o tensor não tem ordem espacial!
Soma com Positional Embeddings: $X = X_{\text{emb}} + W_{\text{pos}}$

Inspetor de Vetor Ativo: "amor" (Posição: t=7 | Token ID: 8954)

$x_7 = W_E[8954] = [+0.882, +0.310, -0.149, +0.592, \dots, +0.621] \in \mathbb{R}^{768}$

Tudo o que você precisa é de amor



Janela de Contexto: Quantidade de Tokens por Solicitação

A Matriz de Entrada $[T \times d_{\text{model}}]$ e o Paradigma da Completação Autoregressiva

Janela de Contexto: Quantidade de Tokens Processados por Solicitação

Tensor de Entrada: [Batch, T=4, d=768]

Complexidade Atenção: $O(T^2)$

1. Matriz da Sequência no Contexto Atual

Cada **linha** é um token no tempo (t); cada **coluna** é uma dimensão de embedding (d).

context = 2 3 4

Posição (t)	Token	dim 0	dim 1	dim 2	dim 3	...	dim 767
$t = 0$	Eu	+0.521	-0.184	+0.342	+0.120	...	+0.612
$t = 1$	sei	+0.153	+0.772	-0.291	+0.803	...	+0.612
$t = 2$	que	+0.341	+0.012	-0.518	+0.194	...	+0.612
$t = 3$	nada	-0.412	+0.225	+0.681	-0.052	...	+0.612

Linhas ativas = 4 tokens

← $d_{\text{model}} = 768$ features contínuas por token →

Contexto Completo: Todos os 4 tokens da frase "Eu sei que nada" cabem simultaneamente na janela de processamento.

2. O Paradigma da Completação

A pergunta do Transformer: "Qual a completção desse texto?"

Próximo Token ($t=4$)

Texto na Janela de Entrada:

Eu sei que nada → "sei"

Distribuição de Probabilidades Softmax:

$P(x_4 | x_0 \dots x_3)$

1. "sei" (Mais Provável)	94.2%
2. "disse"	2.8%
3. "vale"	1.6%
4. "resta"	0.9%
5. "outros"	0.5%

GPT-2: 1.024 tokens

LLaMA 3: 8.192 tokens

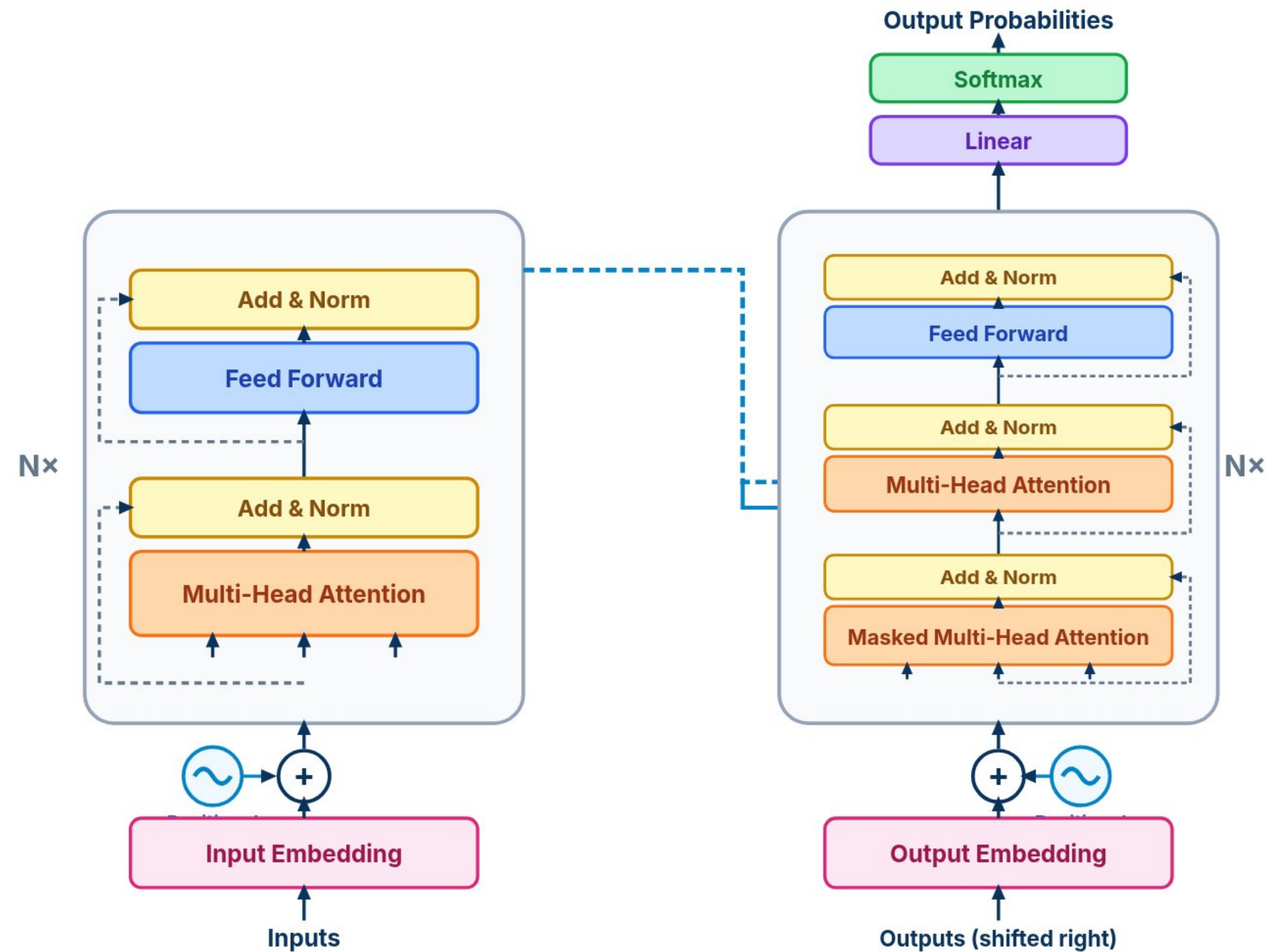
GPT-4o: 128k tokens

Mapa da Arquitetura: Onde Estamos na Rede Transformer?

Consolidando a Base de Entrada e Situando o Salto para a Autoatenção (Q, K, V)

 A Macro-Arquitetura Completa do Transformer (Vaswani et al., 2017)

Attention Is All You Need • NeurIPS 2017





Injeção de Posição: A Soma de Token Embedding + Position Embedding

Como o Transformer Diferencia Palavras Idênticas em Posições Distintas ($X = E_{\text{token}} + E_{\text{pos}}$)

Injeção de Posição: Soma Elemento a Elemento (Token + Posição)

Comparando: "CAT" (pos 1 vs pos 5)

$X = E_{\text{token}} + E_{\text{pos}}$

Tokens Sequência textual	YOUR	CAT	IS	A	LOVELY	CAT
	105	4242	6892	1516	72	4242
Input IDs Posição no vocabulário	207.952 8405.545 1448.853 ...	141.171 3350.276 8191.192 ...	659.621 1051.304 0.656 ...	562.776 5288.567 95.582 ...	6693.422 6080.315 9778.258 ...	141.171 3350.276 8191.192 ...
Embedding Vetor de dimensão 512 / 768	1.856 259.671	3421.633 8473.390	7805.679 4025.506	2194.716 5949.119	2081.141 714.357	3421.633 8473.390
	+	+	+	+	+	+
	1120.415 4512.890 7841.112 ...	1608.664 8133.080 2399.620 ...	5412.100 2190.450 8914.320 ...	3205.110 6540.890 1420.500 ...	4510.900 1890.340 5620.780 ...	1458.281 7890.902 3102.821 ...
Position Emb. Coordenada da posição pos	3210.450 6541.200	9405.386 3159.120	1120.950 4310.880	7890.120 2150.340	3410.900 9810.120	1217.659 7620.018
	=	=	=	=	=	=
Encoder Input Sinal de entrada X	1328.367 12918.435 9289.965 ...	1749.835 11483.356 10590.812 ...	6071.721 3241.754 8914.976 ...	3767.886 11829.457 1516.082 ...	11204.322 7970.655 15399.038 ...	1599.452 11241.178 11294.013 ...
	3212.306 6800.871	12827.019 11632.510	8926.629 8336.386	10084.836 8099.459	5492.041 10524.477	4639.292 16093.408

Por Que Somar em Vez de Concatenar?

A soma elemento a elemento mantém a dimensão `d_model` constante (sem inflar os parâmetros da atenção) e o hiperespaço de 768 dimensões é amplo o bastante para comportar semântica e posição sem interferência destrutiva!

CAT(pos 1) ≠ CAT(pos 5)

Comparativo: Encodings Senoidais vs Embeddings Aprendidos

Vaswani et al. (2017) vs GPT-2 / BERT



Embeddings Senoidais Fixos (Vaswani 2017)

Determinístico & Sem Parâmetros

- Fórmulas trigonométricas analíticas: $PE(pos, 2i) = \sin(...)$ e $PE(pos, 2i+1) = \cos(...)$.
- Não adiciona nenhum parâmetro treinável ao modelo.
- Permite, em teoria, extrapolar para sequências mais longas que as vistas no treino.
- Relação de posição relativa $PE(pos + k)$ pode ser expressa como transformação linear de $PE(pos)$.

🚀 Usado no Transformer original (Attention Is All You Need).



Embeddings Aprendidos (GPT-2, BERT, LLaMA)

Treinável via Backpropagation

- Matriz de parâmetros livres $W_{pos} \in \mathbb{R}^{(T_{max} \times d_{model})}$ inicializada aleatoriamente.
- O modelo aprende autonomamente as representações de distância ideais para o domínio.
- Implementação direta e limpa em PyTorch com `nn.Embedding(max_positions, d_model)`.
- Desempenho empírico idêntico ou superior em tarefas de linguagem natural e visão.

🚀 Padrão adotado na maioria dos LLMs modernos como GPT-2 e BERT.

⚡ Na prática de Deep Learning moderna, tanto funções senoidais quanto tabelas aprendidas entregam excelente performance!

A Mecânica do Scaled Dot-Product: Dimensões e Matriz de Atenção (6 × 6)

T = 6 tokens | d_k = 512

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

$$\text{softmax}\left(\frac{\begin{matrix} Q \\ (6, 512) \end{matrix} \times \begin{matrix} K^T \\ (512, 6) \end{matrix}}{\sqrt{512} \approx 22.63}\right) =$$

Por que o resultado é [6, 6]?
Ao multiplicar a matriz de Queries [6 × 512] pela transposta de Keys [512 × 6], a dimensão interna 512 é eliminada, gerando os scores de afinidade entre todas as 6 palavras com todas as 6 palavras!

[6, 6]

Q \ K	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.23899	0.11911	0.13134	0.15092	0.18093	0.15593	1
CAT	0.14242	0.27829	0.20799	0.12935	0.15534	0.11192	1
IS	0.14470	0.13211	0.26622	0.21902	0.21874	0.12503	1
A	0.14660	0.12899	0.20684	0.21314	0.11814	0.19724	1
LOVELY	0.21000	0.15794	0.15712	0.14537	0.27277	0.17407	1
CAT	0.19559	0.01144	0.20309	0.10455	0.18342	0.30191	1

Cada linha representa uma distribuição de probabilidade **Softmax** (soma = 1) indicando quanto cada palavra atende às demais.



Understanding Attention: A Matemática do Scaled Dot-Product

Passo a Passo Rigoroso: $Q \cdot K^T$, Escala $1/\sqrt{d_k}$, Softmax e Multiplicação por V

Understanding Attention: A Matemática do Scaled Dot-Product

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

Passo 1

Passo 2

Passo 3

Passo 4

FASE ATIVA DE EXECUÇÃO

1. Produto Escalar Matricial ($Q \times K^T$)

$$S = Q \cdot K^T$$

Dimensões dos Tensores: $(B, T, d_k) \times (B, d_k, T) \rightarrow (B, T, T)$

Calcula o produto escalar entre todas as Queries e todas as Keys. Cada elemento $S[i, j]$ representa a afinidade/similaridade bruta não normalizada entre o token i e o token j .

✓ Implementação nativa com `torch.bmm` (Batch Matrix Multiplication) ou `torch.matmul`.

RASTREAMENTO DE FORMATOS DE TENSORES EM PYTORCH

Símbolo	Significado	Shape
Q	Queries projetadas	$[B, T, d_k]$
K^T	Keys transpostas	$[B, d_k, T]$
$Q K^T$	Scores de afinidade	$[B, T, T]$
A (Softmax)	Pesos de atenção	$[B, T, T]$
$A \times V$	Saída contextualizada	$[B, T, d_v]$

⚠ **O Gargalo Quadrático:** A matriz intermédia $Q K^T$ tem dimensão $T \times T$. Se $T=10.000$, essa matriz tem $100.000.000$ elementos por cabeça e batch!

Laboratório Interativo 2: Simulador Matemático de Scaled Dot-Product

Calcule Tensores, Altere dk e Comprove a Função Estabilizadora de $1/\sqrt{dk}$

Laboratório Matemático: Scaled Dot-Product Attention Simulator

Dimensão \$d_k\$:

41664

Escala $1/\sqrt{dk}$: ATIVADA ($\div 4.0$)

1. MATRIZ DE SCORES (SCALED: $Q K^T / \sqrt{DK}$)

dk = 16

Q \ K	"Tudo"	"precisa"	"é"	"amor"
"Tudo"	0.26	0.07	-0.15	-0.32
"precisa"	0.41	0.11	-0.25	-0.51
"é"	0.37	0.09	-0.24	-0.49
"amor"	0.17	0.02	-0.15	-0.28

✓ Scores com variância controlada próxima de 1.0.

2. PESOS DE ATENÇÃO FINAIS A = SOFTMAX(...)

Soma da linha = 100%

Token i	"Tudo"	"precisa"	"é"	"amor"	Σ
"Tudo"	32.9%	27.1%	21.6%	18.3%	100%
"precisa"	37.6%	27.8%	19.5%	15.0%	100%
"é"	36.8%	27.8%	19.9%	15.5%	100%
"amor"	30.9%	26.8%	22.6%	19.8%	100%

💡 **Conclusão:** Sem a escala $1/\sqrt{d_k}$, com $d_k=64$, o Softmax vira quase uma função degrau (0% / 100%), matando os gradientes de todas as outras posições durante o treino!

Visualizing Attention: A Matriz de Atenção em Ação

Inspeção do Mapa de Calor $T \times T$ e Relações Semânticas na Letra dos Beatles

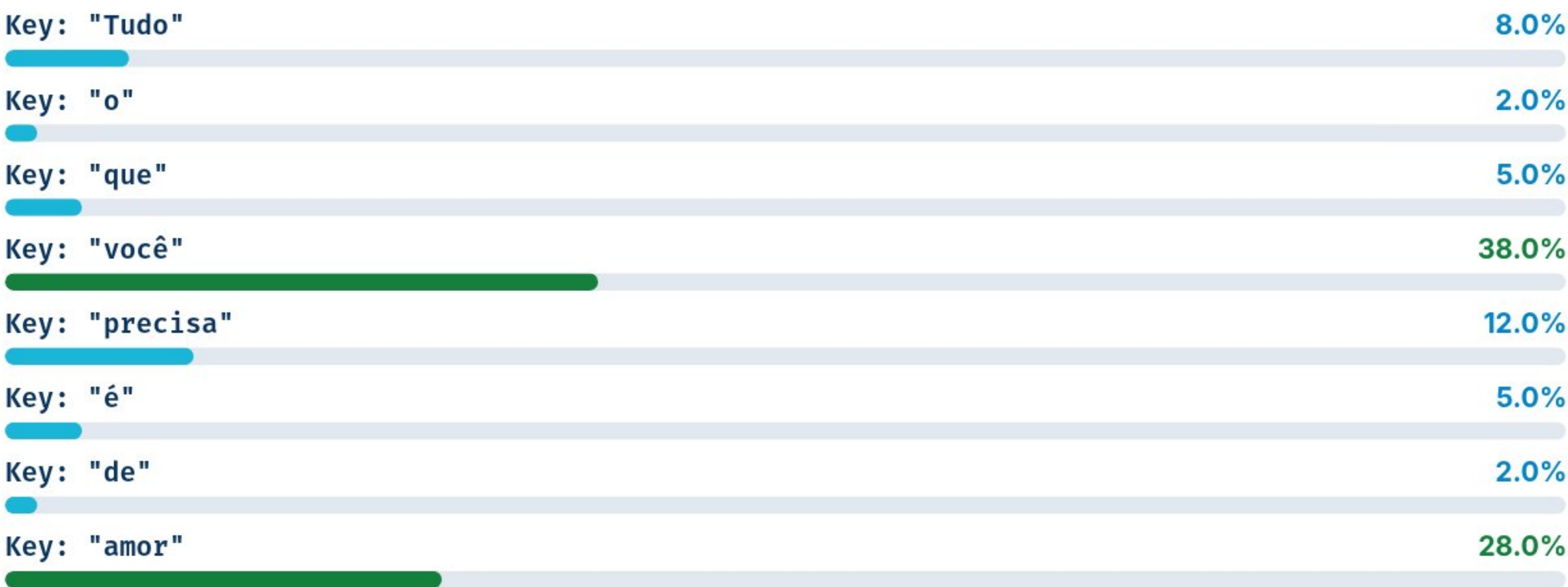
Visualizing Attention: A Matriz de Atenção em Ação ($T \times T$)

Atenção Bidirecional (BERT/ViT)

SELECIONE O TOKEN QUERY NA CANÇÃO:

"Tudo" "o" "que" "você" "precisa" "é" "de" "amor"

Para onde a Query "precisa" está prestando atenção?



Interpretação: O modelo aprende padrões sintáticos (sujeito-verbo) e semânticos (objeto-verbo) sem nenhuma regra gramatical explícita injetada manualmente!

MAPA DE CALOR (MATRIZ 8×8)

Matriz Completa $N \times N$

Q\K	Tud	o	que	voc	pre	é	de	amo
Tud	.45	.10	.05	.05	.15	.05	.03	.12
o	.15	.50	.20	.05	.04	.02	.01	.03
que	.10	.20	.40	.15	.10	.02	.01	.02
voc	.05	.02	.10	.35	.40	.02	.01	.05
pre	.08	.02	.05	.38	.12	.05	.02	.28
é	.20	.02	.03	.05	.10	.40	.10	.10
de	.02	.01	.02	.02	.25	.08	.20	.40
amo	.25	.02	.03	.10	.32	.08	.05	.15

Custo de Atenção: Cada bloco Transformer calcula essa matriz $T \times T$ para cada cabeça de atenção em cada camada de forma totalmente paralela na GPU.

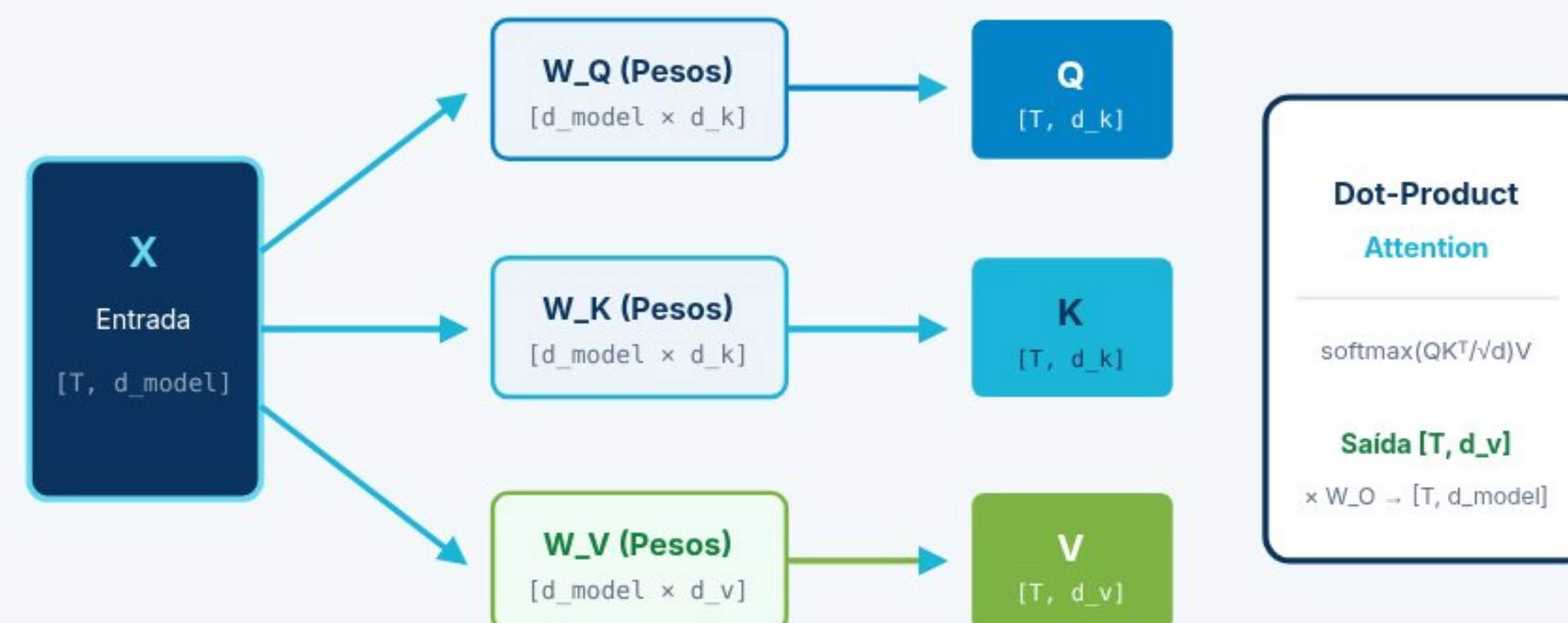
Tornando a Atenção Treinável: As Matrizes de Projeção W_Q , W_K , W_V

Transformações Lineares Aprendíveis e Propagação de Gradientes em PyTorch

🔧 Tornando a Atenção Treinável: As Matrizes de Projeção W_Q , W_K , W_V , W_O

`nn.Linear(d_model, d_k, bias=False)`

PROJEÇÕES LINEARES APRENDÍVEIS A PARTIR DO INPUT X



Implementação PyTorch

```
class SelfAttention(nn.Module):
    def __init__(self, d_model, d_k):
        super().__init__()
        self.W_q = nn.Linear(d_model, d_k, bias=False)
        self.W_k = nn.Linear(d_model, d_k, bias=False)
        self.W_v = nn.Linear(d_model, d_k, bias=False)
        self.W_o = nn.Linear(d_k, d_model, bias=False)
        self.d_k = d_k

    def forward(self, x):
        Q = self.W_q(x) # (B, T, d_k)
        K = self.W_k(x) # (B, T, d_k)
        V = self.W_v(x) # (B, T, d_k)
        scores = (Q @ K.transpose(-2, -1)) / math.sqrt(self.d_k)
        A = torch.softmax(scores, dim=-1)
        out = A @ V # (B, T, d_k)
        return self.W_o(out)
```

✅ **Treinamento de Ponta a Ponta:** Os parâmetros em W_Q , W_K , W_V , W_O são ajustados pelo otimizador AdamW através do cálculo padrão de gradientes com perda Cross-Entropy.

Causal Masking: Atenção Autorregressiva Sem Vazamento de Futuro

A Máscara Triangular Inferior e o Truque de $-\infty$ no Softmax

Causal Masking: Impedindo o Modelo de "Espiar o Futuro"

```
scores.masked_fill(mask == 0, float('-inf'))
```

MATRIZ DE ATENÇÃO CAUSAL PÓS-SOFTMAX (8 × 8)

Triângulo Superior Bloqueado (0.0)

Pos	Tud	o	que	voc	pre	é	de	amo
1. Tud	1.00	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2. o	.50	.50	0.0	0.0	0.0	0.0	0.0	0.0
3. que	.33	.33	.33	0.0	0.0	0.0	0.0	0.0
4. voc	.25	.25	.25	.25	0.0	0.0	0.0	0.0
5. pre	.20	.20	.20	.20	.20	0.0	0.0	0.0
6. é	.17	.17	.17	.17	.17	.17	0.0	0.0
7. de	.14	.14	.14	.14	.14	.14	.14	0.0
8. amo	.13	.13	.13	.13	.13	.13	.13	.13

Posições em **vermelho (0.0)** foram pré-preenchidas com $-\infty$ antes do Softmax, garantindo que $\exp(-\infty) / \sum = 0.0$.



Por Que a Máscara é Obrigatória?

- Geração Autoregressiva:** Durante a inferência, os tokens futuros *ainda não foram gerados*.
- Treinamento em Paralelo:** Durante o treino, passamos a sequência inteira de uma vez para máxima velocidade na GPU, mas usamos a máscara causal para que o modelo não "trapaceie" olhando a resposta certa à frente.
- Construto em PyTorch:** Criamos uma máscara triangular inferior usando `torch.tril(torch.ones(T, T))` e registramos no módulo com `register_buffer`.

Máscara Causal em PyTorch:

```
mask = torch.tril(torch.ones(T, T))
scores = scores.masked_fill(mask == 0, -float('inf'))
A = torch.softmax(scores, dim=-1) # Zeros no futuro!
```

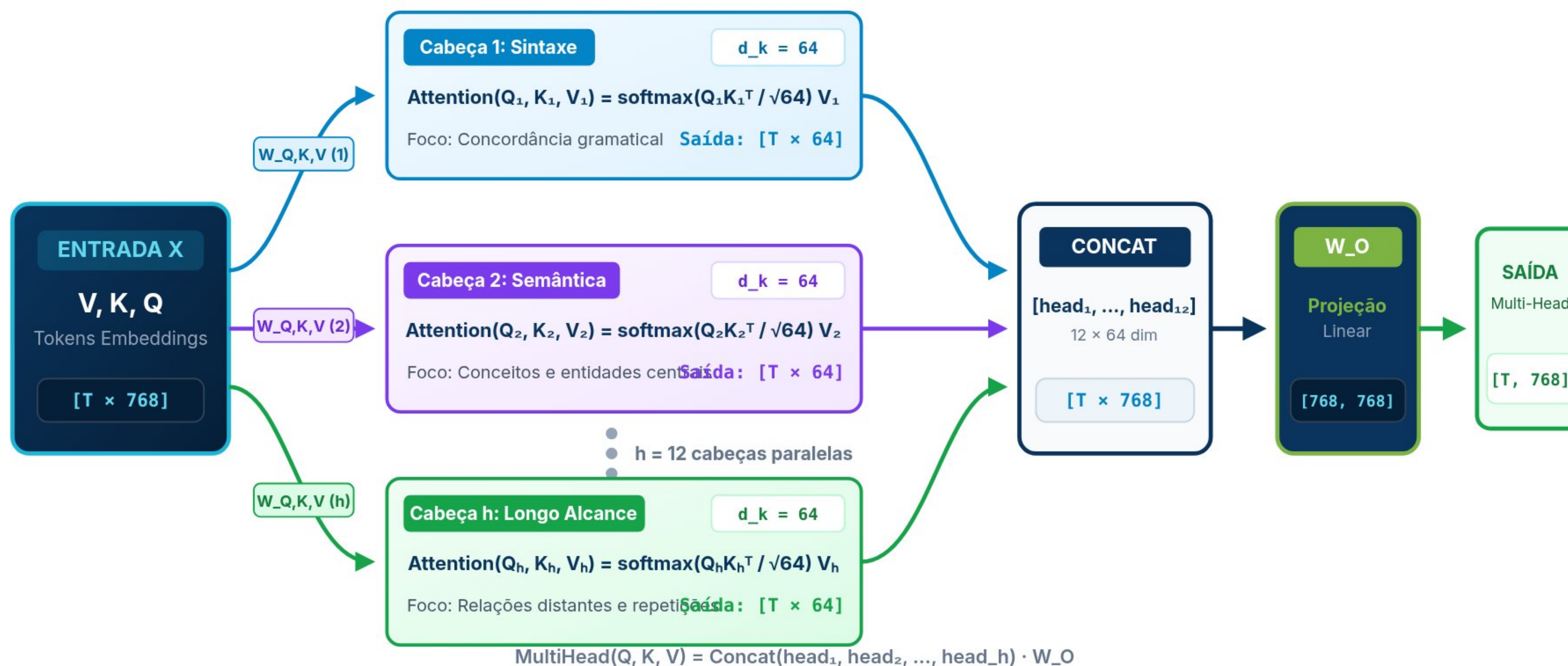

Multi-Head Attention: Múltiplos Subespaços de Representação

Por Que Uma Cabeça Não Basta? Divisão em h Cabeças e Projeção WO

Arquitetura Multi-Head Attention: Divisão em h Cabeças e Projeção Linear WO

$$d_k = d_{\text{model}} / h = 768 / 12 = 64$$

Entrada [T, 768] → Saída [T, 768]





Laboratório Interativo 3: Inspetor Visual de Multi-Head Attention

Alterne Entre Cabeças Sintáticas, Semânticas e Rítmicas na Canção dos Beatles

Laboratório: Inspetor de Múltiplas Cabeças na Canção dos Beatles

12 Cabeças no GPT-2 / ViT-Base

Cabeça 1: Sintaxe & Concordância

Cabeça 2: Negação & Escopo Semântico

Cabeça 3: Métrica & Estrutura Poética

Cabeça 4: Janela Local (Bigramas)

FRASE: "(AMOR) NÃO HÁ NADA QUE VOCÊ POSSA FAZER QUE NÃO POSSA SER FEITO"

não há nada que você possa fazer que não possa ser feito

CONEXÕES ATIVAS NESTA CABEÇA:

- você ↔ possa (sujeito-verbo) 92%
- possa ↔ fazer (modal-infinitivo) 88%
- possa ↔ feito (passiva modal) 85%
- fazer ↔ feito (paralelismo verbal) 78%

Esta cabeça rastreia a gramática profunda da oração, conectando o pronome "você" aos verbos que governam sua ação.

DETALHAMENTO DO SUBESPAÇO DA CABEÇA

Foco Semântico: **Relação Sujeito-Verbo e Modais**

Dimensão da Cabeça (\$d_k\$): **64 valores**

Matrizes de Pesos: $W_Q^{(1)}$, $W_K^{(1)}$, $W_V^{(1)}$

Por que isso supera Redes Convolucionais?

Enquanto uma CNN precisa de 10 a 20 camadas para relacionar "você" (palavra 5) a "feito" (palavra 12), a Multi-Head Attention calcula essa ligação na **primeira camada** em apenas 1 operação matricial!

A Arquitetura Original para Tradução: Encoder-Decoder

Como Vaswani et al. (2017) Conectaram Fonte e Alvo com Cross-Attention

A Arquitetura Original: Encoder-Decoder para Tradução (Vaswani et al., 2017)

1. Encoder (Inglês)

2. Cross-Attention (Ponte)

3. Decoder (Português)

ENCODER (Língua Fonte: Inglês)

Prompt de Entrada: "All you need is love"

All you **need** is love

Self-Attention Bidirecional

Sem máscara — Cada palavra vê todas as outras (Visão Global)

Feed-Forward (MLP) + Residuais + LN

Expansão 4x (768 → 3072 → 768) + GELU

Representações de Contexto Fonte

Matrizes K_{enc} e $V_{enc} \in \mathbb{R}^{(T_{src} \times d_{model})}$

Processado uma única vez em paralelo!

Cross-Attention

Q: do Decoder (PT)

K: do Encoder (EN)

V: do Encoder (EN)

Alinhamento Bilíngue

A "Ponte de Tradução"

DECODER AUTORREGRESSIVO (Língua Alvo: Português)

Prefixo Gerado até o momento:

<BOS> Tudo o que você **precisa** é de amor

1. Masked Self-Attention (Causal)

Máscara Triangular: Impede que a palavra atual veja o futuro em português

2. Cross-Attention (Encoder-Decoder Attention)

Queries (Português: "precisa") atendem para Keys/Values (Inglês: "need")

3. Feed-Forward (MLP) + Linear LM Head + Softmax

Projeção Linear para o Vocabulário em Português ($|V_{pt}| = 50.000$)

Próxima Palavra Prevista com Alta Confiança:

$P(x_t = \text{"amor"} \mid \text{"Tudo o que você precisa é de"}) = 96.8\%$ ✨

Alinhamento de Tradução Ativo: Fonte (EN): "need" → Alvo (PT): "precisa"

💡 Dica: Clique nas palavras em português acima para inspecionar o alinhamento da Cross-Attention!

Variantes da Atenção: Cross-Attention, FlashAttention e Eficiência

Self vs Cross-Attention e Otimizações de Memória SRAM na GPU

Variantes da Atenção: Cross-Attention, FlashAttention e Mecanismos Modernos

Self vs Cross-Attention

FlashAttention (GPU IO-Aware)

1. Self-Attention (Autoatenção)

$$Q = X \cdot W_Q, K = X \cdot W_K, V = X \cdot W_V$$

- **Origem Única:** Queries, Keys e Values derivam da *mesma* sequência de entrada X .
- **Propósito:** Cada token contextualiza seu significado em relação aos outros tokens da mesma frase/imagem.
- **Onde é usado:** No encoder de ViT, nos blocos do GPT e no BERT.

2. Cross-Attention (Atenção Cruzada)

$$Q = X_{dec} \cdot W_Q, K = Y_{enc} \cdot W_K, V = Y_{enc} \cdot W_V$$

- **Duas Sequências Distintas:** Queries vêm do Decoder (X_{dec}); Keys e Values vêm do Encoder (Y_{enc}).
- **Propósito:** Permite que o Decoder consulte representações externas ricas (ex: imagem, texto em outro idioma ou áudio).
- **Onde é usado:** Modelos Multimodais (Flamingo, CLIP, Stable Diffusion onde o texto guia a geração da imagem, DETR na detecção).

A Anatomia do Bloco Transformer: MHA, MLP, Residuais e LayerNorm

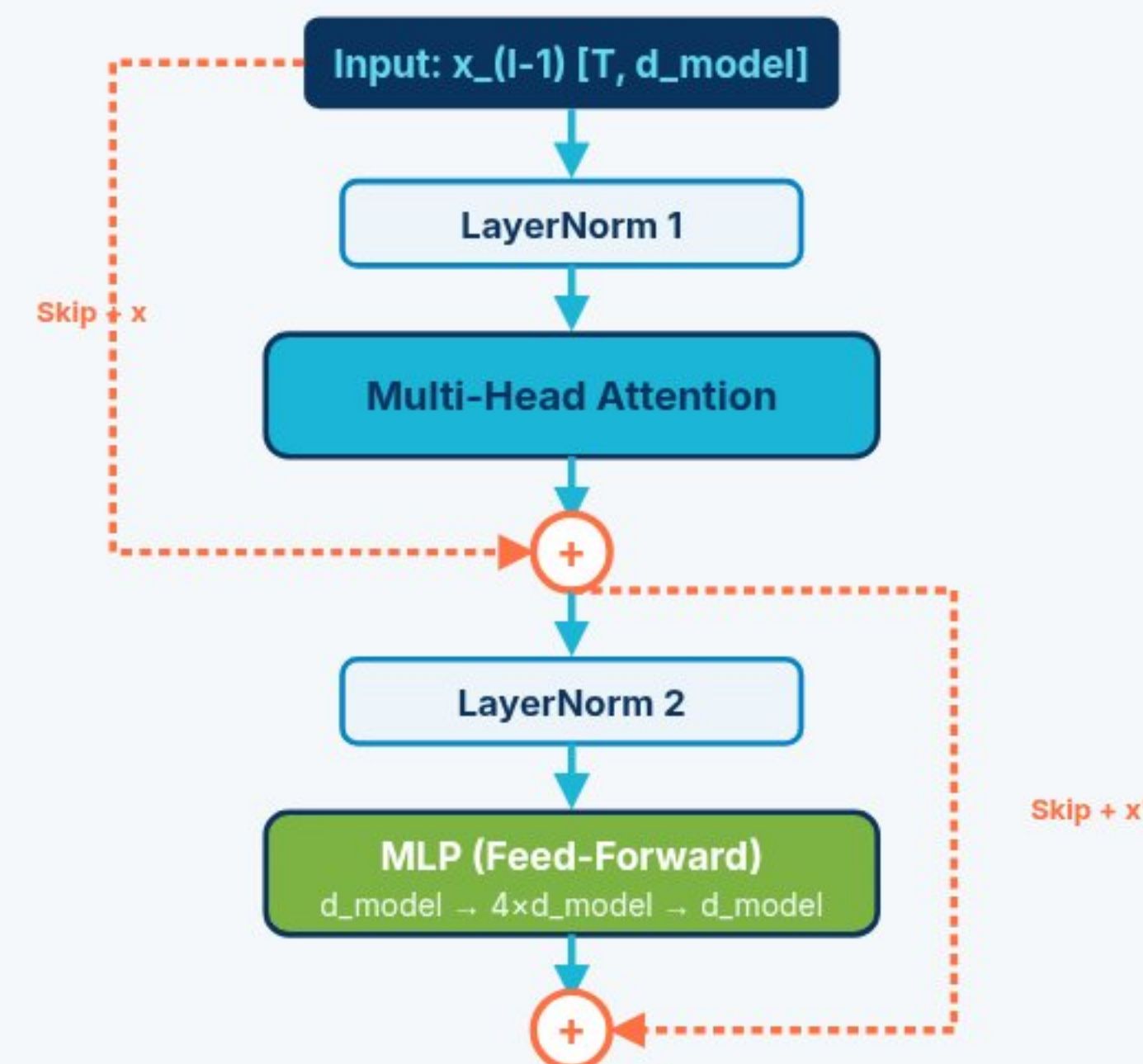
O Padrão Pre-LN, Expansão 4x no Feed-Forward e Ativações GELU

A Anatomia do Bloco Transformer: Atenção + MLP + Residual + LayerNorm

Pre-LN (Padrão Moderno: GPT / ViT)

Post-LN (Vaswani Original 2017)

FLUXO INTERNO (PRE-LAYERNORM)



Os 4 Pilares do Bloco

- **1. LayerNorm:** Normaliza as ativações através da dimensão de features d para média 0 e variância 1 (estabiliza gradientes).
- **2. Multi-Head Attention:** Permite comunicação e mistura dinâmica de informações entre diferentes posições da sequência.
- **3. Skip Connections (Residuais):** Supervias de gradiente direto que possibilitam empilhar 32, 64 ou mais camadas sem degradação.
- **4. MLP Position-wise (Expansão 4x):** Rede linear de 2 camadas com ativação GELU ($768 \rightarrow 3072 \rightarrow 768$). Processa cada token individualmente, funcionando como uma "memória associativa".

✓ **Pre-LN:** Permite treinar modelos profundos sem warm-up agressivo, sendo o padrão adotado no GPT-2, GPT-3 e Vision Transformer.

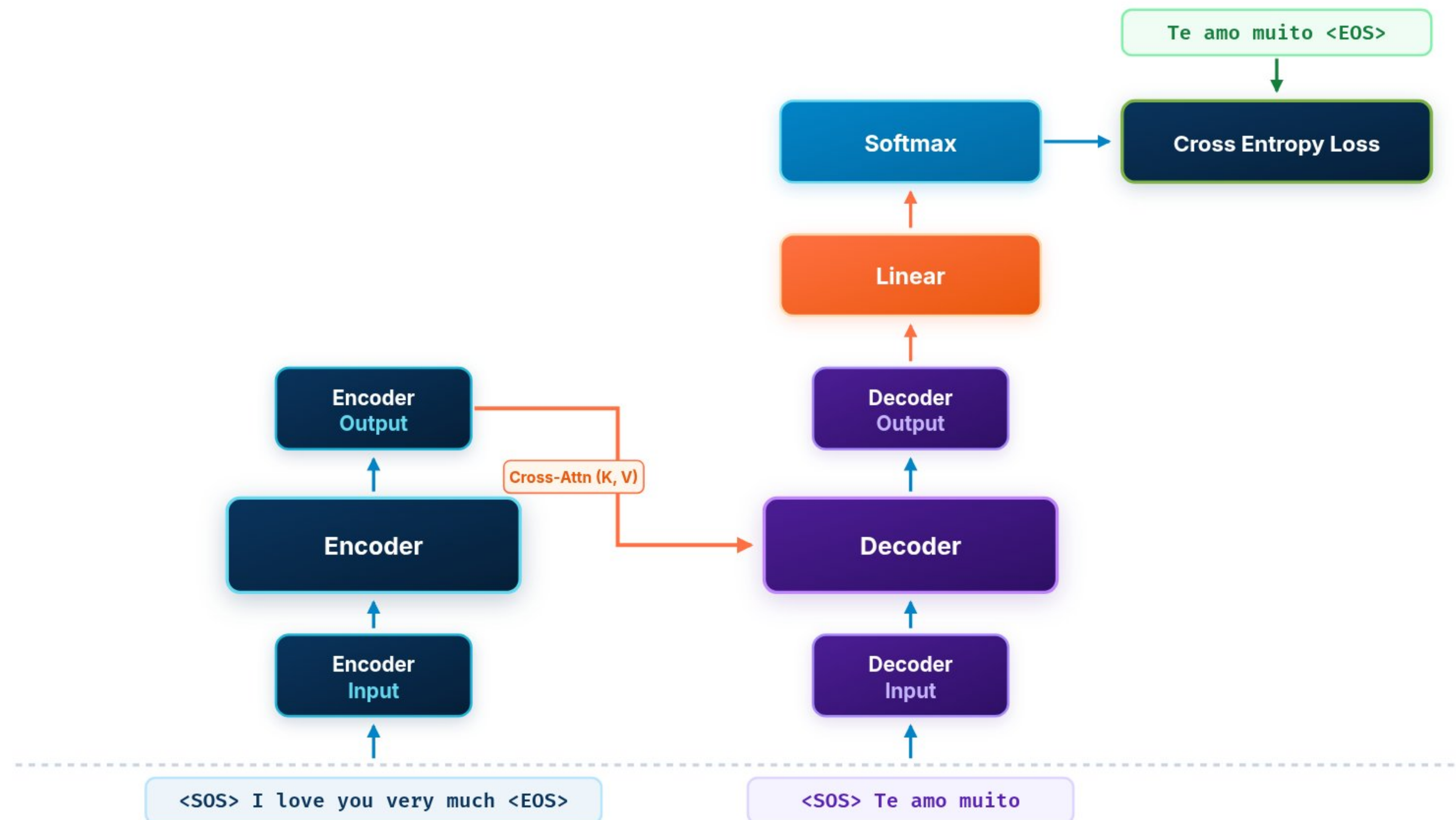
O Transformer Tudo Junto: Macro-Arquitetura de Ponta a Ponta

O Fluxo de Execução e Treinamento Seq2Seq: Encoder, Decoder, Softmax e Loss

Macro-Arquitetura Completa de Ponta a Ponta: Fluxo de Execução e Treinamento

Seq2Seq Encoder-Decoder

Supervised Training Loop





Laboratório Interativo 4: Simulador de Parâmetros e Complexidade

Ajuste Camadas, Dimensões e Contexto para Estimar Parâmetros e VRAM da GPU

Simulador de Trade-offs: Parâmetros, Memória VRAM & Atenção $O(T^2)$

Presets: GPT-2 Small (124M) GPT-2 Medium (350M) BERT Base (110M) Transformer (2017)

HIPERPARÂMETROS DA ARQUITETURA

Dimensão do Embedding (d_{model}):

768

Número de Camadas (N):

12 blocos

Cabeças de Atenção (h):

12 cabeças ($d_k = 64$)

Janela de Contexto (T):

1024 tokens / patches

Batch Size (B):

8

ESTIMATIVA DE RECURSOS E MEMÓRIA DA GPU

Parâmetros Totais

124.4M

Attn: 28.3M | MLP: 56.6M

VRAM de Treino (Adam FP32)

1.85 GB

Pesos puros FP16: 237.2 MB

Custo de Ativações dos Mapas $T \times T$:

2304.0 MB / batch

Dimensão interna de cada cabeça (d_k):

64

GFLOPs estimados por token:

0.25 GFLOPs

Insight Prático: 2/3 de todos os parâmetros de cada bloco residem na rede Feed-Forward (MLP $4 \times d_{model}$), enquanto a maior parte da memória volátil no forward vem das matrizes de atenção $T \times T$.

Laboratório Interativo 5: Quiz de Fixação de Conhecimentos

Avalie Seu Domínio Sobre BPE, Scaled Dot-Product, Máscaras e Bloco Transformer

? Laboratório 5: Quiz de Fixação de Conhecimentos em Transformers

Questão 1 de 5

1. Por que os Transformers modernos utilizam o algoritmo Byte-Pair Encoding (BPE) em vez de tokenização por palavras inteiras?

Porque o BPE reduz a dimensão do vetor de embedding para apenas 8 bits.

Porque elimina palavras fora do vocabulário (OOV) mantendo um vocabulário fixo e compacto através da fusão estatística de subpalavras e bytes.

Porque o BPE calcula a atenção diretamente no texto bruto sem precisar de matrizes de projeção linear.

Porque redes convolucionais só conseguem processar caracteres individuais.